

```
!pip install gym==0.26.2
!pip install numpy==1.26.4
```

```
import gym
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from collections import deque
import random
import matplotlib.pyplot as plt # Optional: still used if you want inline plots
import json
import os
```

```
FAST_MODE = False # Set to False for more thorough training
```

```
# Model save path:
MODEL_PATH = "dqn_taxi_model.h5"
METRICS_PATH = "dqn_taxi_metrics.json"

def build_model(state_size, action_size, learning_rate=0.001):
    """
    Build a simple feedforward DNN to approximate the Q-function.
    The input dimension corresponds to the state vector,
    and the output gives the Q-value for each possible action.
    """
    #####
    ### To-Do (12 points):
    ### Implement a DNN with input size as state_size and output size as action_size
    ### Use Dense layers with ReLU activations; final layer should have action_size outputs
    ### Use Adam optimizer and mean squared error as the loss function
    # #env = gym.make("Taxi-v3")

    model = keras.Sequential()
    model.add(layers.Dense(128, input_dim=state_size, activation="relu"))
    model.add(layers.Dense(128, activation="relu"))
    model.add(layers.Dense(action_size, activation="linear"))

    model.compile(loss="mse", optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate))
    #####

    return model
```

```
def preprocess_state(state, state_size):
    """
    Convert a discrete integer state (0-499 for Taxi-v3)
    into a one-hot encoded vector suitable for the neural network.
    """
    state = int(state)
    state_one_hot = np.zeros((state_size,), dtype=np.float32)
    state_one_hot[state] = 1.0
    return np.reshape(state_one_hot, [1, state_size]) # For model input
```

```
def choose_action(state, model, action_size, epsilon):
    """
    Use epsilon-greedy policy:
    - With probability epsilon, select a random action (exploration)
    - Otherwise, select the best predicted Q-value (exploitation)
    """
    ### To-Do (8 points):

    if np.random.rand() <= epsilon: #with probability epsilon or greater, the model will act randomly so have it cho
        return random.randrange(action_size)
    else:
        q_values = model.predict(state, verbose=0)
        return np.argmax(q_values[0])

    # exploit vs. explore
```

```

def get_hyperparameters(fast_mode=True):
    """
    Return a configuration dictionary with training hyperparameters.
    FAST_MODE uses fewer episodes for quick testing.
    FULL_MODE performs longer, more stable training.
    """
    if fast_mode:
        return {
            "mode": "FAST",
            "episodes": 200,
            "max_steps": 100,
            "batch_size": 64,
            "gamma": 0.99,
            "epsilon_start": 1.0,
            "epsilon_min": 0.05,
            "epsilon_decay": 0.995,
            "replay_buffer_size": 2000,
            "target_update_freq": 10,
            "learning_rate": 0.001,
        }
    else:
        return {
            "mode": "FULL",
            "episodes": 1000,
            "max_steps": 200,
            "batch_size": 64,
            "gamma": 0.99,
            "epsilon_start": 1.0,
            "epsilon_min": 0.01,
            "epsilon_decay": 0.995,
            "replay_buffer_size": 10000,
            "target_update_freq": 10,
            "learning_rate": 0.0005,
        }

```

```

def train_dqn(env, fast_mode=True):
    """
    Train a DQN agent to learn an optimal policy in the Taxi-v3 environment.
    The agent uses experience replay, target networks, and epsilon decay.
    """
    state_size = env.observation_space.n
    action_size = env.action_space.n

    config = get_hyperparameters(fast_mode=fast_mode)

    model = build_model(state_size, action_size, learning_rate=config["learning_rate"])
    target_model = build_model(state_size, action_size, learning_rate=config["learning_rate"])
    target_model.set_weights(model.get_weights()) # Initialize target model weights

    # Hyperparameters
    episodes = config["episodes"]
    max_steps = config["max_steps"]
    batch_size = config["batch_size"]
    gamma = config["gamma"]
    epsilon = config["epsilon_start"]
    epsilon_min = config["epsilon_min"]
    epsilon_decay = config["epsilon_decay"]
    replay_buffer = deque(maxlen=config["replay_buffer_size"])
    target_update_freq = config["target_update_freq"]

    # Track performance metrics
    rewards_per_episode = []
    epsilon_history = []
    steps_per_episode = []
    loss_per_episode = []

    for episode in range(episodes):
        state, _ = env.reset()
        state = preprocess_state(state, state_size)
        total_reward = 0
        done = False
        steps = 0
        losses_this_episode = []

```

```

for step in range(max_steps):
    # Choose an action
    action = choose_action(state, model, action_size, epsilon)

    # Execute the action in the environment
    next_state, reward, terminated, truncated, _ = env.step(action)
    done = terminated or truncated
    next_state_processed = preprocess_state(next_state, state_size)

    # Store experience in replay buffer
    replay_buffer.append((state, action, reward, next_state_processed, done))
    state = next_state_processed
    total_reward += reward
    steps += 1

    # Update the network if enough samples are collected
    if len(replay_buffer) >= batch_size:
        minibatch = random.sample(replay_buffer, batch_size)

        # Vectorized batch training
        states_mb = np.vstack([m[0] for m in minibatch])
        actions_mb = np.array([m[1] for m in minibatch])
        rewards_mb = np.array([m[2] for m in minibatch])
        next_states_mb = np.vstack([m[3] for m in minibatch])
        dones_mb = np.array([m[4] for m in minibatch])

        ### To-Do (24 points):
        # Compute Q-value targets using Bellman equation
        #####
        ### Implement the pseudo-code provided in the background under training):
        ### - Calculate predicted Q-values
        ### - Update the target value using the Bellman equation
        ### - Train the model
        #####
        q_values = model.predict(states_mb, verbose=0)
        q_next = target_model.predict(next_states_mb, verbose=0)
        max_next_q = np.max(q_next, axis=1)

        targets = q_values.copy()

        for i in range(batch_size):
            if dones_mb[i]:
                target_q = rewards_mb[i]
            else:
                target_q = rewards_mb[i] + gamma * max_next_q[i]
            targets[i, actions_mb[i]] = target_q

        history = model.fit(states_mb, targets, epochs = 1, verbose=0)
        losses_this_episode.append(history.history["loss"][0])

    if done:
        break

# Record episode metrics
rewards_per_episode.append(total_reward)
steps_per_episode.append(steps)
avg_loss = float(np.mean(losses_this_episode)) if losses_this_episode else None
loss_per_episode.append(avg_loss)

# Update target network weights periodically
if (episode + 1) % target_update_freq == 0:
    target_model.set_weights(model.get_weights())

# Decay epsilon
if epsilon > epsilon_min:
    epsilon *= epsilon_decay
epsilon_history.append(float(epsilon))

# Progress logging
if avg_loss is not None:
    print(f"Episode {episode + 1}/{episodes}, Reward: {total_reward}, Steps: {steps}, Loss: {avg_loss:.4f}")
else:
    print(f"Episode {episode + 1}/{episodes}, Reward: {total_reward}, Steps: {steps}, Loss: N/A")

print("Training completed.")

```

```

metrics = {
    "rewards": rewards_per_episode,
    "epsilons": epsilon_history,
    "steps": steps_per_episode,
    "losses": loss_per_episode,
}

return model, metrics, config

```

```

def plot_training_results_matplotlib(metrics):
    """
    Quick visualization using matplotlib for debugging or class demos.
    """
    #####
    ### To-Do (8 points):
    ### Create two plots:
    ### 1. Total rewards per episode
    ### 2. Epsilon decay over episodes
    #####

    rewards = metrics["rewards"]
    epsilons = metrics["epsilons"]

    episodes = range(1, len(rewards) + 1)

    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.plot(episodes, rewards)
    plt.xlabel("Episode")
    plt.ylabel("Total Rewards")
    plt.title("Total Rewards per Episode")

    plt.subplot(1, 2, 2)
    plt.plot(episodes, epsilons)
    plt.xlabel("Episode")
    plt.ylabel("Epsilon Decay")
    plt.title("Epsilon Decay over Episodes")

    plt.tight_layout()
    plt.show()

```

```

def evaluate_model(model, env, state_size, episodes=10, max_steps=200):
    """
    Evaluate the model by running episodes without exploration (epsilon=0).
    Reports average reward and average steps.
    """
    total_rewards = 0
    total_steps = 0

    for episode in range(episodes):
        state, _ = env.reset()
        state = preprocess_state(state, state_size)
        done = False
        episode_reward = 0
        steps = 0

        while not done and steps < max_steps:
            #####
            ### To-Do (12 points):
            ### Select the best action and evaluate the model
            #####
            q_values = model.predict(state, verbose=0)
            action = np.argmax(q_values[0])

            next_state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated

            episode_reward += reward
            steps += 1

            state = preprocess_state(next_state, state_size)

        total_rewards += episode_reward

```

```

    total_steps += steps
    print(f"Evaluation Episode {episode + 1}: Reward = {episode_reward}, Steps = {steps}")

avg_reward = total_rewards / episodes
avg_steps = total_steps / episodes if episodes > 0 else 0.0
print(f"Average Evaluation Reward over {episodes} episodes: {avg_reward:.2f}")
print(f"Average Steps per Evaluation Episode: {avg_steps:.2f}")

return {"avg_reward": float(avg_reward), "avg_steps": float(avg_steps), "episodes": episodes}

```

```

def generate_html_report(metrics, config, filename="dqn_taxi_report.html", eval_stats=None):
    """
    Generate an interactive HTML dashboard showing training curves
    (rewards, epsilon, loss, steps) and a summary of efficiency gains.
    """

    rewards = metrics["rewards"]
    epsilons = metrics["epsilons"]
    steps = metrics["steps"]
    losses = metrics["losses"]

    # Replace None in losses with null for valid JSON/JS
    losses_js = ["null" if v is None else v for v in losses]

    # Efficiency summary: early vs late episodes
    n = len(rewards)
    if n > 0:
        window = max(1, n // 5) # use ~20% of episodes for early/late windows
        avg_reward_start = float(sum(rewards[:window]) / window)
        avg_reward_end = float(sum(rewards[-window:]) / window)
        avg_steps_start = float(sum(steps[:window]) / window)
        avg_steps_end = float(sum(steps[-window:]) / window)
    else:
        window = 0
        avg_reward_start = avg_reward_end = 0.0
        avg_steps_start = avg_steps_end = 0.0

    reward_change = avg_reward_end - avg_reward_start
    steps_change = avg_steps_start - avg_steps_end # positive if steps decreased

    eval_avg_reward = eval_stats["avg_reward"] if eval_stats is not None else None
    eval_avg_steps = eval_stats["avg_steps"] if eval_stats is not None else None
    eval_episodes = eval_stats["episodes"] if eval_stats is not None else None

    ### To-Do (26 points):
    ### HTML CODE GOES HERE ###
    html = f"""<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>DQN Taxi-v3 Training Report</title>
<script src="https://cdnjs.cloudflare.com/ajax/libs/plotly.js/2.18.0/plotly.min.js"></script>
<style>
    body {{
        padding: 20px;
    }}
    .container{{
        display: flex;
    }}
    h1{{
        text-align: center;
    }}
    .main{{
        flex: 2;
        padding: 0px;
        margin-right: 0px;
        margin-left: 100px;
    }}
    .sidebar{{
        flex: 1;
        width: 225px;
        padding-left: 20px;
    }}
    .card {{
        background: #f5f5f5;

```

```

padding: 10px;
margin: 20px 0;
border-radius: 8px;
}}
.chartgrid{{
    display: grid;
    grid-template-columns: 2fr 2fr;
    gap: 5px;
    padding: 0px;
    margin-top: 0;
}}
.chart {{
    min-width: 400px;
    height: auto;
}}
</style>
</head>
<body>
<h1>DQN Taxi-v3 Training Report</h1>
<div class="container">
    <div class="sidebar">
        <div class="card">
            <h2>Run Configuration</h2>
            <ul style="list-style-type: none; padding: 0;">
                <li><strong>Mode:</strong> {config['mode']}</li>
                <li><strong>Episodes:</strong> {config['episodes']}</li>
                <li><strong>Batch Size:</strong> {config['batch_size']}</li>
                <li><strong>Replay Buffer Size:</strong> {config['replay_buffer_size']}</li>
                <li><strong>Learning Rate:</strong> {config['learning_rate']}</li>
                <li><strong>Target Update Freq:</strong> {config['target_update_freq']}</li>
            </ul>
        </div>
        <div class="card">
            <h2>Training Progress</h2>
            <ul style="list-style-type: none; padding: 0;">
                <li><strong>Early Avg Reward:</strong> {avg_reward_start:.2f}</li>
                <li><strong>Late Avg Reward:</strong> {avg_reward_end:.2f}</li>
                <li><strong>Change:</strong> {reward_change:+.2f}</li>
                <li><strong>Early Avg Steps:</strong> {avg_steps_start:.2f} → </li>
                <li><strong>Late Avg Steps:</strong> {avg_steps_end:.2f}</li>
                <li><strong>Reduction:</strong> {steps_change:+.2f}</li>
                <li><strong>Evaluation (No Exploration):</strong> </li>
                <li>{f'Avg Reward = {eval_avg_reward:.2f}, Avg Steps = {eval_avg_steps:.2f}' if eval_stats else ''}</li>
            </ul>
        </div>
    </div>
    <div class="main">
        <div class="chartgrid">
            <div class="chart">
                <h4>1. Total Reward per Episode</h4>
                <div id="rewards"></div>
            </div>
            <div class="chart">
                <h4>2. Epsilon (Exploration Rate) Decay</h4>
                <div id="epsilon"></div>
            </div>
            <div class="chart">
                <h4>3. Average Loss per Episode</h4>
                <div id="loss"></div>
            </div>
            <div class="chart">
                <h4>4. Steps per Episode</h4>
                <div id="steps"></div>
            </div>
        </div>
    </div>
</div>
<script>
    Plotly.newPlot('rewards', [{
        x: {list(range(1, len(rewards) + 1))},
        y: {rewards},
        type: 'scatter',
        mode: 'lines+markers',
        line: {{color: 'green'}}
    }], {{xaxis: {{title: 'Episode'}}, yaxis: {{title: 'Total Reward'}}, margin: {{t: 20}} }});
</script>

```

```

Plotly.newPlot('epsilon', [{
    x: {list(range(1, len(epsilons) + 1))},
    y: {epsilons},
    type: 'scatter',
    mode: 'lines+markers',
    line: {{color: 'blue'}}
}],
{{xaxis: {{title: 'Episode'}}},
yaxis: {{title: 'Epsilon'}}},
margin: {{t: 20}}
});

Plotly.newPlot('loss', [{
    x: {list(range(1, len(losses) + 1))},
    y: {losses_js},
    type: 'scatter',
    mode: 'lines+markers',
    line: {{color: 'purple'}}
}],
{{xaxis: {{title: 'Episode'}}},
yaxis: {{title: 'Loss'}}},
margin: {{t: 20}}
});

Plotly.newPlot('steps', [{
    x: {list(range(1, len(steps) + 1))},
    y: {steps},
    type: 'scatter',
    mode: 'lines+markers',
    line: {{color: 'orange'}}
}],
{{xaxis: {{title: 'Episode'}}},
yaxis: {{title: 'Steps'}}},
margin: {{t: 20}}
});
</script>
</body>
</html>
""""

# filename="dqn_taxi_report.html" # This line is redundant if filename is already a parameter
report_path = os.path.abspath(filename)
with open(report_path, "w", encoding="utf-8") as f:
    f.write(html)

import webbrowser
print(f"HTML report written to: {report_path}")
print("Open this file in your browser to view the interactive dashboard.")
webbrowser.open(f"file://{report_path}")

from google.colab import files
files.download("/content/dqn_taxi_report.html")

pass

if __name__ == "__main__":
    env = gym.make("Taxi-v3")
    state_size = env.observation_space.n

    # filename = "dqn_taxi_report.html" # This is now passed as a default arg to generate_html_report

    RETRAIN = True

    if not RETRAIN and os.path.exists(MODEL_PATH) and os.path.exists(METRICS_PATH):
        print("Found saved model and metrics. Loading from disk instead of retraining.")
        model = tf.keras.models.load_model(MODEL_PATH, custom_objects={'mse': tf.keras.losses.MeanSquaredError})
        with open(METRICS_PATH, "r", encoding="utf-8") as f:
            saved = json.load(f)
            metrics = saved["metrics"]
            config = saved["config"]
    else:
        print("No saved model found. Training a new DQN agent.")
        model, metrics, config = train_dqn(env, fast_mode=FAST_MODE)
        model.save(MODEL_PATH)
        with open(METRICS_PATH, "w", encoding="utf-8") as f:
            json.dump({"metrics": metrics, "config": config}, f)
        print(f"Model saved to {os.path.abspath(MODEL_PATH)}")

```

```
print(f"Metrics saved to {os.path.abspath(METRICS_PATH)}")

eval_stats = evaluate_model(model, env, state_size, episodes=10)
generate_html_report(metrics, config, filename="dqn_taxi_report.html", eval_stats=eval_stats)

env.close()
```

```
from google.colab import files
files.download("/content/dqn_taxi_report.html")
```